Shadow Trace

Analyse a suspicious file, uncover hidden clues, and trace the source of the infection.

Task 1 - Scenario

It's the middle of the night shift. You're the only analyst in the SOC when a manager calls in urgently: a suspicious file was found on a user's machine and needs **immediate review**.

You open the file and start digging. Something doesn't look normal for a company updater, and at the same time, the EDR throws a couple of alerts.

Your task: analyse the file, collect anything to identify it, gather any potential IOCs, correlate and analyse the alerts for potential malicious behaviour. It's up to you to piece together what's happening before it spreads further.

Learning Objectives

- Extract IOCs from suspicious binaries
- Correlate alerts with malicious activity
- Perform basic SOC triage actions

Task 2 – File analysis

Analyse the binary located C:\Users\DFIRUser\Desktop\windows-update.exe in the attached machine, answer the questions below.

You can find several tools installed in the machine that can help you with any kind of analysis under C:\Users\DFIRUser\DFIR Tools

Answer the questions below

What is the architecture of the binary file windows-update.exe?

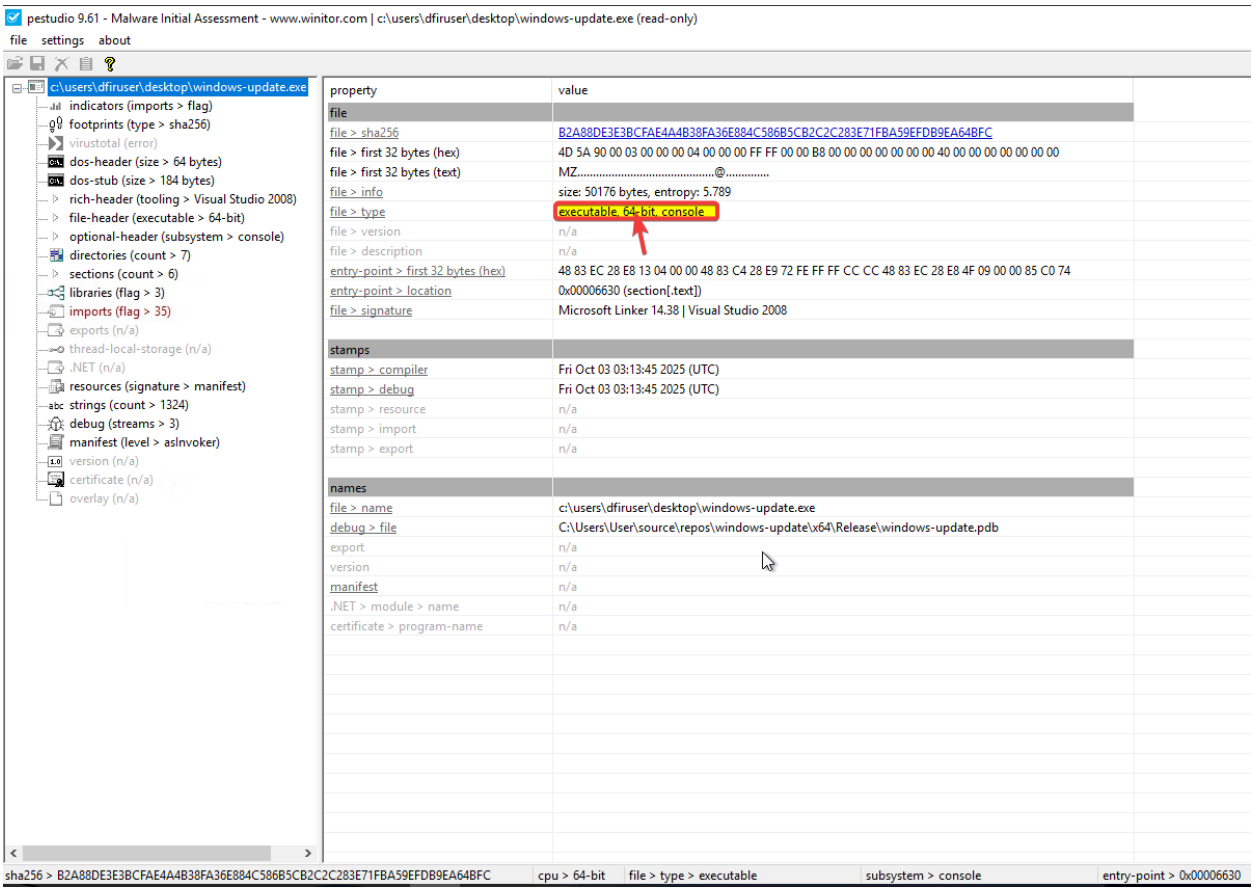

Figure 1 - PEStudio showing windows-update.exe as a 64-bit console executable.

I opened “windows-update.exe” in PEStudio to review the basic PE metadata. PEStudio identified the file as a “64-bit console executable”, which confirms the binary architecture.

Answer: 64-bit

What is the hash (sha-256) of the file windows-update.exe?

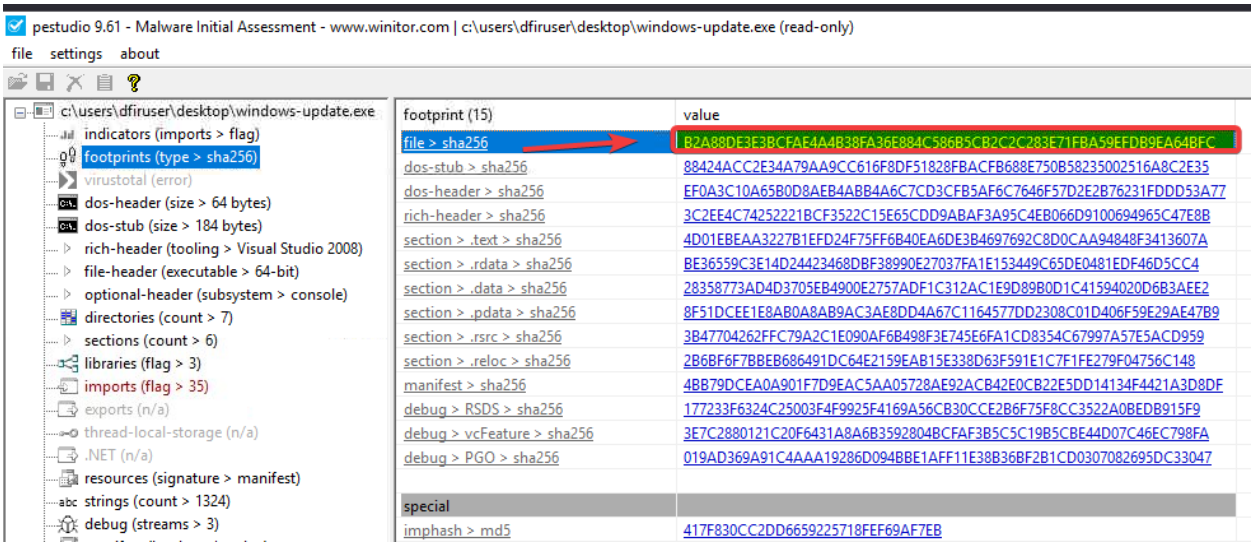

Figure 2 - PEStudio showing the SHA-256 hash of windows-update.exe.

I used PEStudio's footprint section to collect the SHA-256 hash of "windows-update.exe". This hash can be used as an indicator of compromise and to identify the same sample in other tools or environments.

Answer: b2a88de3e3bcfae4a4b38fa36e884c586b5cb2c2c283e71fba59efdb9ea64bfc

Identify the URL within the file to use it as an IOC

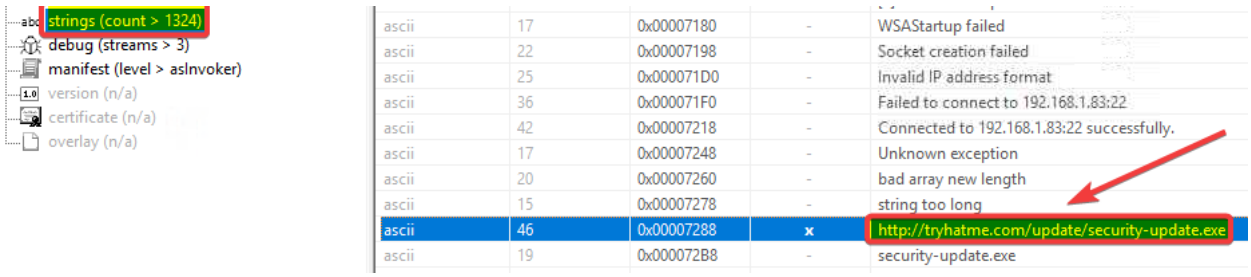

Figure 3 - PEStudio strings showing the embedded URL used as an IOC.

I reviewed the strings in PEStudio and found a suspicious embedded URL inside "windows-update.exe". This URL appears to point to another executable named "security-update.exe", making it a useful IOC for further investigation.

Answer: URL IOC: http://tryhatme.com/update/security-update.exe

With the URL identified, can you spot a domain that can be used as an IOC?

```
PS C:\Users\DFIRUser\Desktop> strings .\windows-update.exe | Select-String -Pattern "http|\.com"

Sysinternals - www.sysinternals.com
http://tryhatme.com/update/security-update.exe
tryhatme.com/VEhNe3lvdV9nMHRfc29tZV9JT0NzX2ZyaWVuZH0=
responses.tryhatme.com
[!] HttpOpenRequestA failed.
[!] HttpSendRequestA failed.
[+] Hosts file exfiltrated to http://
HttpOpenRequestA
HttpSendRequestA
```

Figure 4 - strings.exe output showing the domain responses.tryhatme.com inside the binary.

After finding the embedded URL in PEStudio, I used strings.exe to extract possible URLs and domains from “windows-update.exe”. Filtering the strings output for “http” and “.com” revealed another domain, “response.tryhatme.com”, which can be used as an IOC.

Command used:

strings .\windows-update.exe | Select-String -Pattern "http|\.com"

Answer: responses.tryhatme.com

Input the decoded flag from the suspicious domain

```
PS C:\Users\DFIRUser\Desktop> strings .\windows-update.exe | Select-String -Pattern "tryhatme.com/*"

http://tryhatme.com/update/security-update.exe
tryhatme.com/VEhNe3lvdV9nMHRfc29tZV9JT0NzX2ZyaWVuZH0=
responses.tryhatme.com
```

Figure 5 - strings.exe output showing the Base64-encoded value inside the suspicious URL.

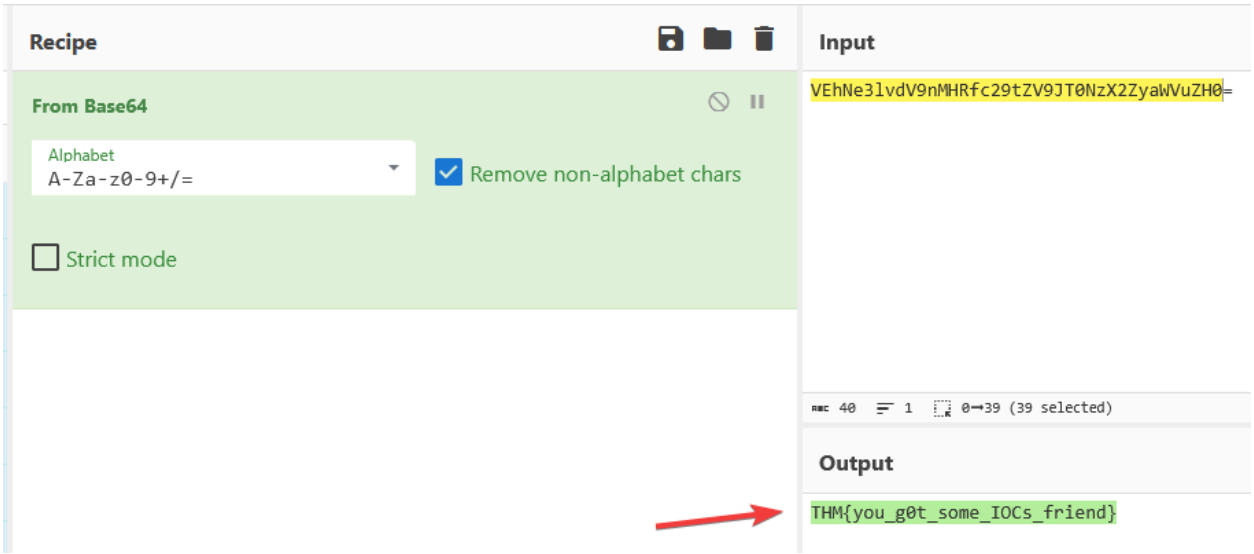

Figure 6 - CyberChef decoding the Base64 value and revealing the flag.

After extracting URLs from “windows-update.exe” using strings.exe, I noticed that one of the URLs contained a Base64-encoded value in the path. I copied only the encoded string and decoded it in CyberChef using the “From Base64” recipe. The decoded output revealed the flag.

Command used:

strings .\windows-update.exe | Select-String -Pattern "tryhatme.com/*"

Encoded value:

VEhNe3lvdV9nMHRfc29tZV9jTzZyaWVuZHo=

Decoded flag:

Answer: THM{you_g0t_some_IOCs_friend}

What library related to socket communication is loaded by the binary?

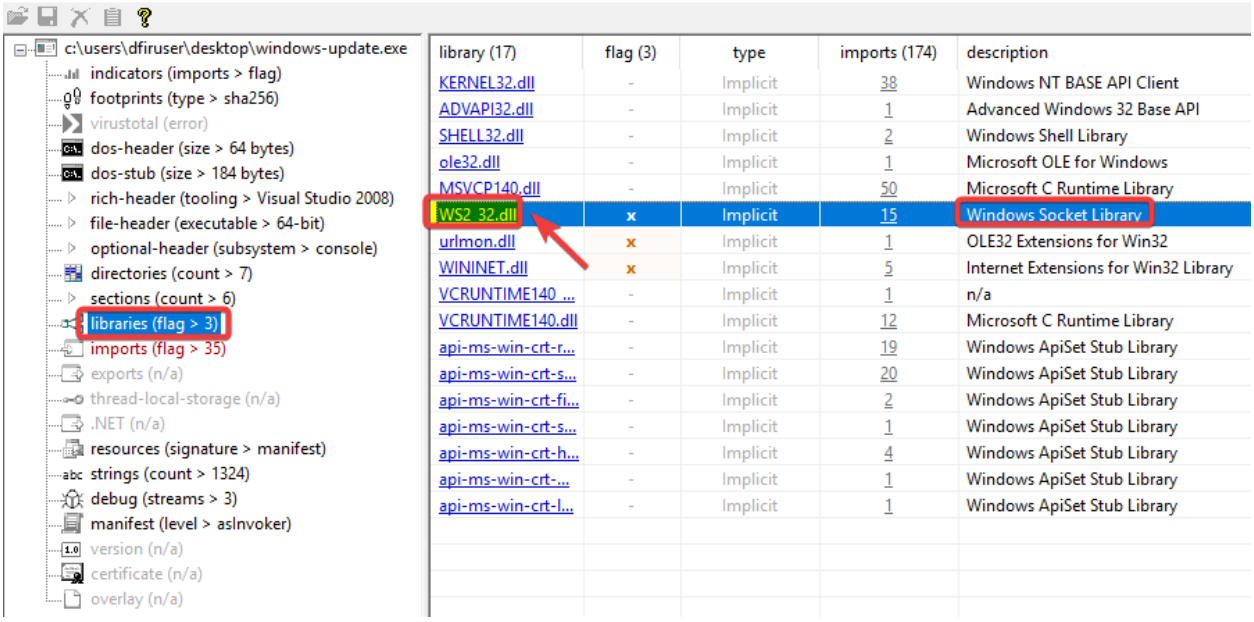

Figure 7 - PEStudio showing WS2_32.dll loaded by the binary.

I reviewed the imported libraries in PEStudio and found that the binary loads “WS2_32.dll”. This Windows library is commonly used for socket communication, which supports the earlier findings that the binary contains embedded network indicators.

Answer: WS2_32.dll

Task 3 – Alerts Analysis

Click on the **View Site** button attached to this task to display the static site in split view. Review the alternatives and answer the questions below.

Answer the questions below

Can you identify the malicious URL from the trigger by the process powershell.exe?

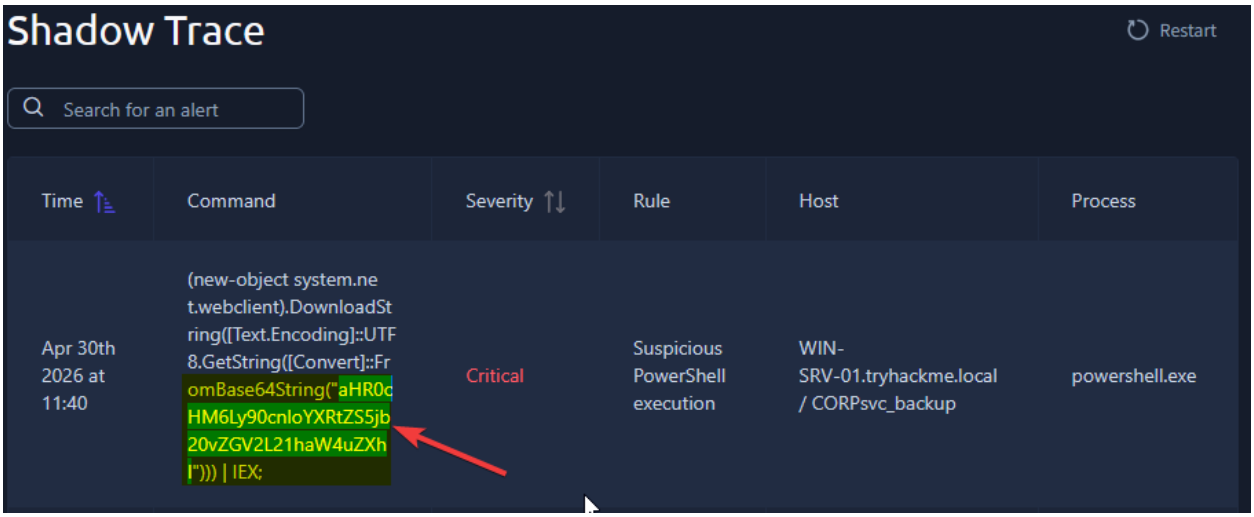

Figure 8 - Alert dashboard showing the Base64-encoded URL triggered by powershell.exe.

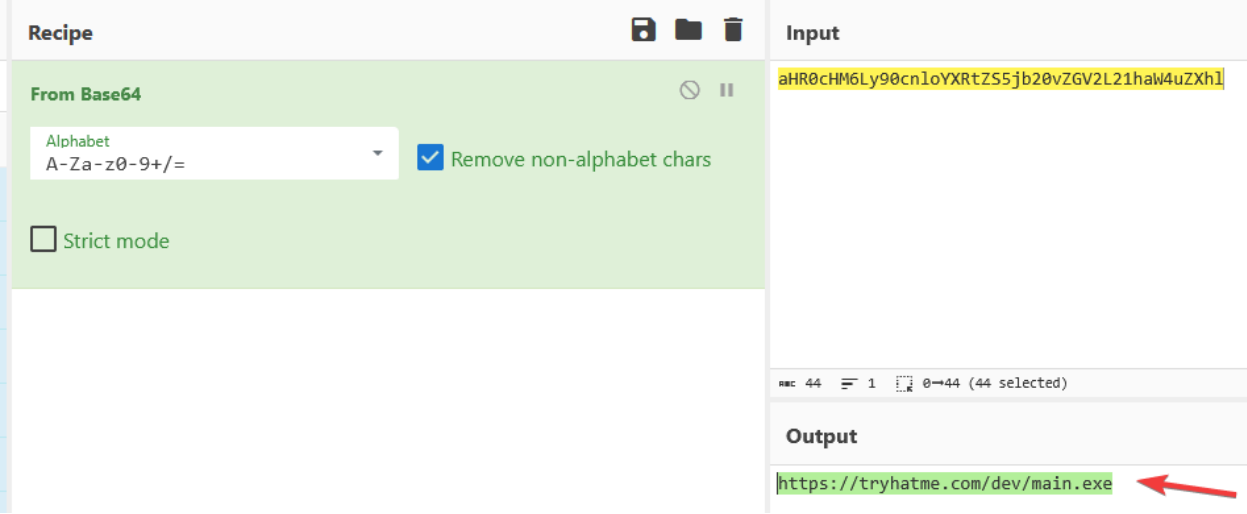

Figure 9 - CyberChef decoding the PowerShell Base64 value and revealing the malicious URL.

In the alert dashboard, I reviewed the command executed by “powershell.exe”. The command contained a Base64-encoded string inside a download cradle. After decoding the Base64 value in CyberChef, it revealed the malicious URL used by the PowerShell process.

Encoded value:

aHR0cHM6Ly90cnloYXRtZS5jb20vZGV2L21haW4uZXhl

Decoded URL:

Answer: https://tryhatme.com/dev/main.exe

Can you identify the malicious URL from the alert triggered by chrome.exe?

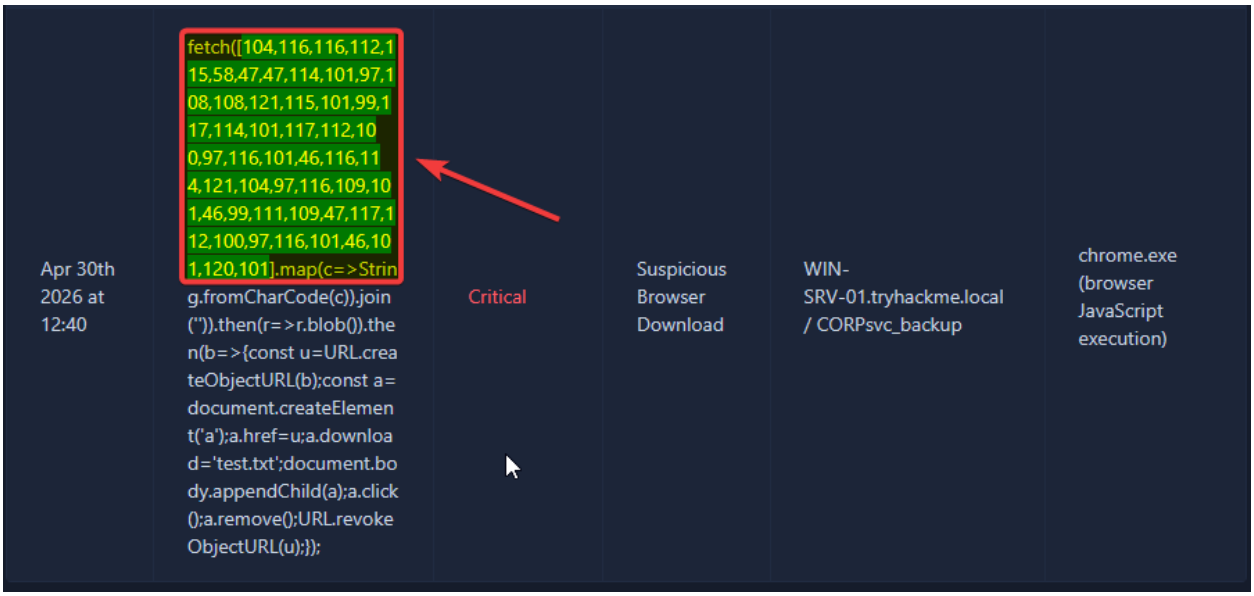

Figure 10 - Shadow Trace alert dashboard showing the chrome.exe alert with an obfuscated JavaScript download command.

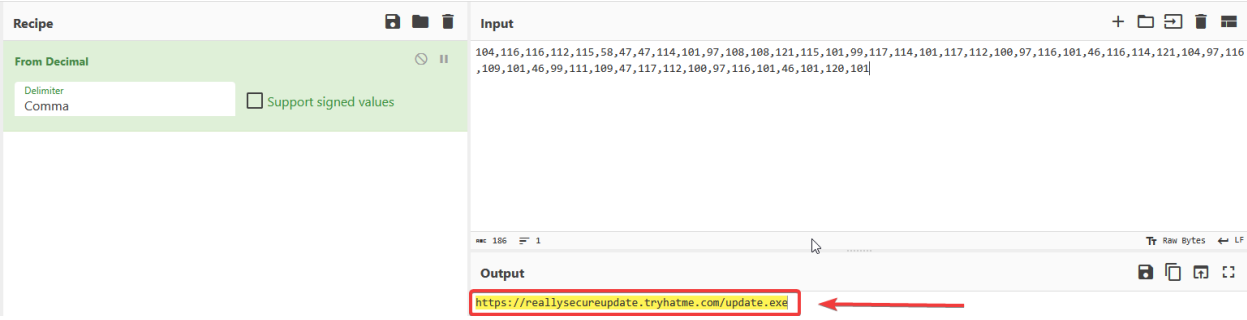

Figure 11 - CyberChef decoding the decimal character codes and revealing the malicious URL.

The alert triggered by “chrome.exe” contained JavaScript code using “String.fromCharCode()” to hide a URL as decimal character codes. I copied the decimal values into CyberChef and used the “From Decimal” recipe with the delimiter set to comma. This converted the numbers back into readable text and revealed the malicious download URL.

Decoded URL:

Answer: https://reallysecureupdate.tryhatme.com/update.exe

What's the name of the file saved in the alert triggered by chrome.exe?

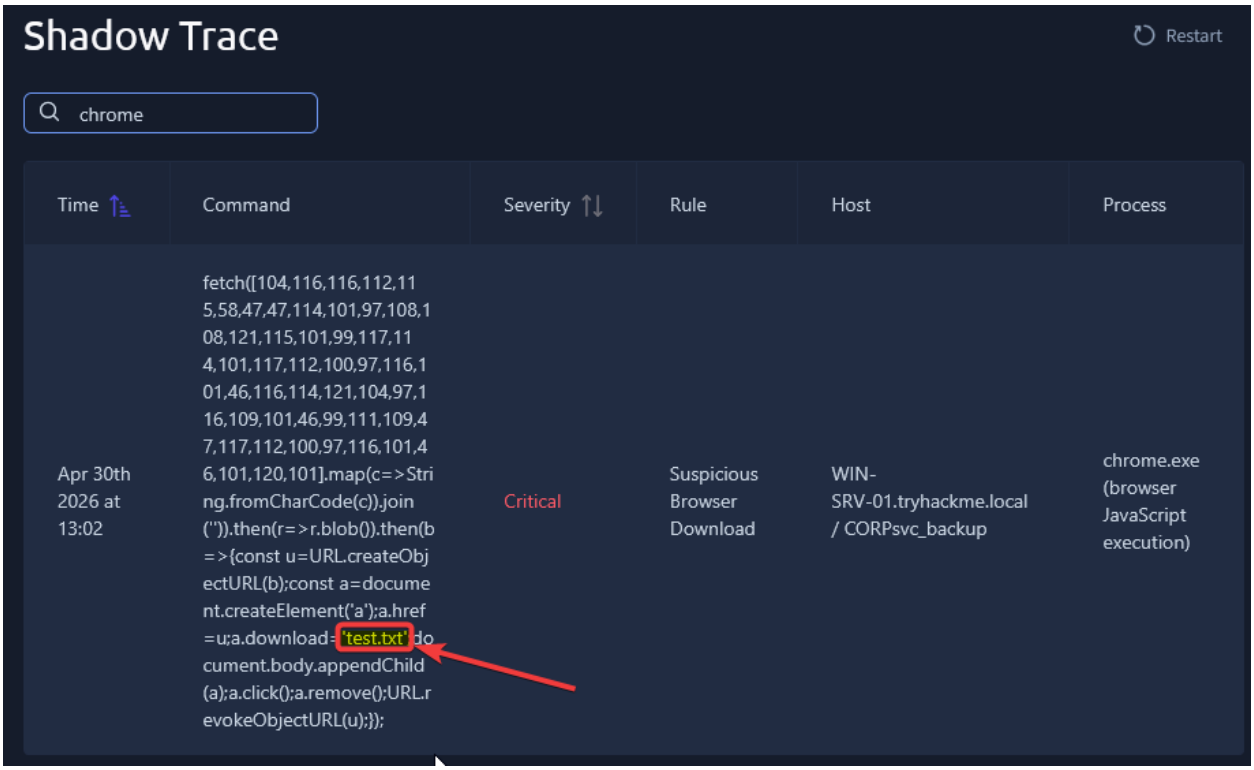

Figure 12 - Shadow Trace alert dashboard showing the filename “test.txt” in the chrome.exe JavaScript download command.

In the alert triggered by “chrome.exe”, the JavaScript creates an anchor element and sets the download filename. The value assigned to “a.download” shows that the file would be saved as “test.txt”.

Saved filename:

Answer: test.txt